OpsPilot

Every operations request read, checked against the systems of record, and routed to the person who can decide it

Agentic AI Operations Assistant for a growing retail chain · take-home for the Forward Deployed Engineer role

Live app https://92-4-93-133.sslip.io

Code github.com/RishabhRaizada/agentic-ops-assistant

Demo login any role on the login page · password Demo@2026

Rishabh Kumar Raizada · 4 November 2026

Ops decisions today depend on inbox archaeology and follow-ups

Urgent Delivery Delay – Store Launch

Our supplier has informed us that 500 display units required for the Pune store launch will be delayed by 10 days. The store is scheduled to open next week, and only 120 units are currently available in the warehouse. Please assess the impact, identify possible alternatives, and obtain approval for any additional cost by 4:00 PM today.

The assignment's sample email: five facts, four systems to check, one approver to find, hours to decide.



Facts are scattered

Inventory, purchase orders, supplier quotes and the store calendar all live in different places; the email is often wrong or incomplete.



Deadlines are hours, not days

A 4 PM cut-off leaves no time to chase people for numbers or for the right signature.



Authority depends on amount and region

Who may approve ₹36k vs ₹1.4 lakh vs ₹3 lakh is tribal knowledge; escalation is manual.



No record of who decided what

Management summaries are rewritten by hand; there is no trail an auditor can trust.

Agents read and propose, code verifies and routes, people decide

AI agents

- Extract facts with an exact evidence quote for each
- Detect duplicates, updates, cancellations
- Assess risk and business impact
- Query systems and propose costed options
- Critique the plan, draft stakeholder emails and management briefs

Deterministic code

- Checks email claims against inventory, POs and store calendar
- Computes shortfall, launch slip, revenue at risk
- Scores confidence from evidence, not self-report
- Picks the approver by amount, region and limit
- Runs SLA timers, escalation, retries and the audit chain

People

- Correct any fact; AI value is kept beside theirs
- Choose, edit or replace an option
- Approve, reject with a budget cap, or ask a question
- Click send on every outbound message
- Close with a resolution reason

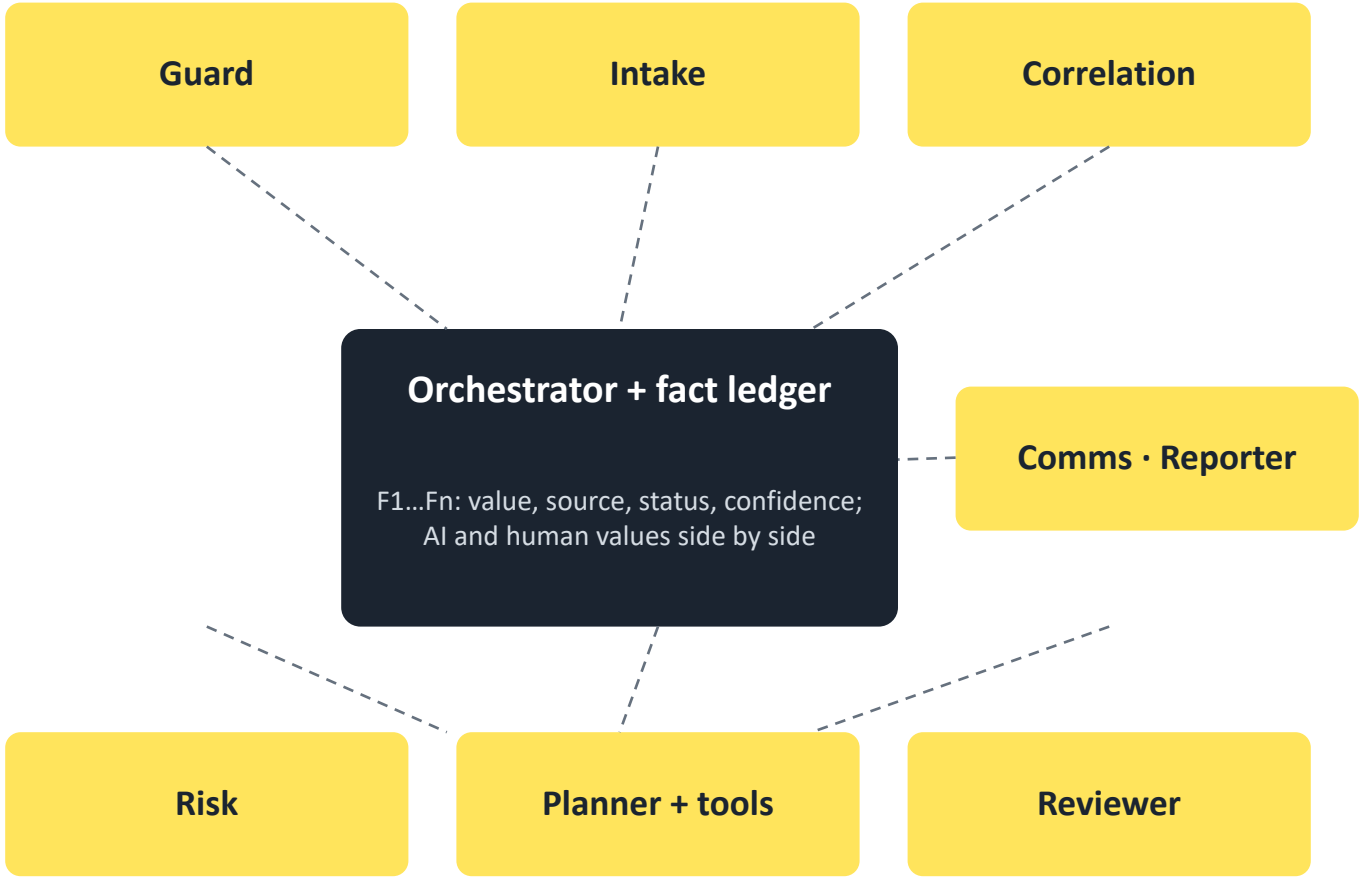
From inbox to closure in one tracked flow



1	2	3	4	5	6	7	8	9
Receive	**Screen**	**Extract**	**Correlate**	**Verify**	**Assess**	**Plan**	**Decide**	**Execute & close**
Email, form, .eml or webhook	Injection guard on datamarked text	Facts + evidence quotes	Duplicate, update or new	Inventory, POs, calendar	Risk agent vs rules engine	Tool calls → 3 costed options → review	Policy routing, SLA, escalation	Actions, comms, brief, audit
System	AI agent	AI agent	AI agent	Code	AI agent	AI agent	Person	Person

Assignment email, live: about 25 seconds from receipt to three costed options (₹36k / ₹1.42 lakh / ₹2.89 lakh). The ₹1.42 lakh option is routed to the West regional head with the reasons: additional spend, critical risk, supplier order change. If she does not respond by the due time, it escalates to the director.

Seven agents share one fact ledger, and never call each other



Typed I/O contracts · every claim cites F# or T# · validators force self-correction

Human approval is mandatory when

- any spend, supplier order or customer-commitment change
- risk is high or critical
- confidence is below 0.75, injection is suspected, or AI was unavailable
- any outbound stakeholder email (a person clicks send)

Confidence is earned, not self-reported

field = min(model, grounding)

overall = extraction × agent-vs-rules agreement × 0.7^{n} conflicts × reviewer × citations

Below 0.5 the AI option cannot be submitted without a written justification.

A small, boring production footprint around the agents

React SPA	Vite + TypeScript + Tailwind; role-aware screens; live polling of agent progress
Caddy	Automatic HTTPS, static files, /api reverse proxy, security headers
FastAPI	Auth + RBAC · state machine · policy engine · reconciler · confidence · SLA monitor · outbox · hash-chained audit
PydanticAI agents	7 typed agents, read-only tools, output validators, FallbackModel
Data & systems	SQLite (WAL, Postgres-ready) · inventory, PO, supplier-quote and store-calendar adapters with circuit breakers and fault injection

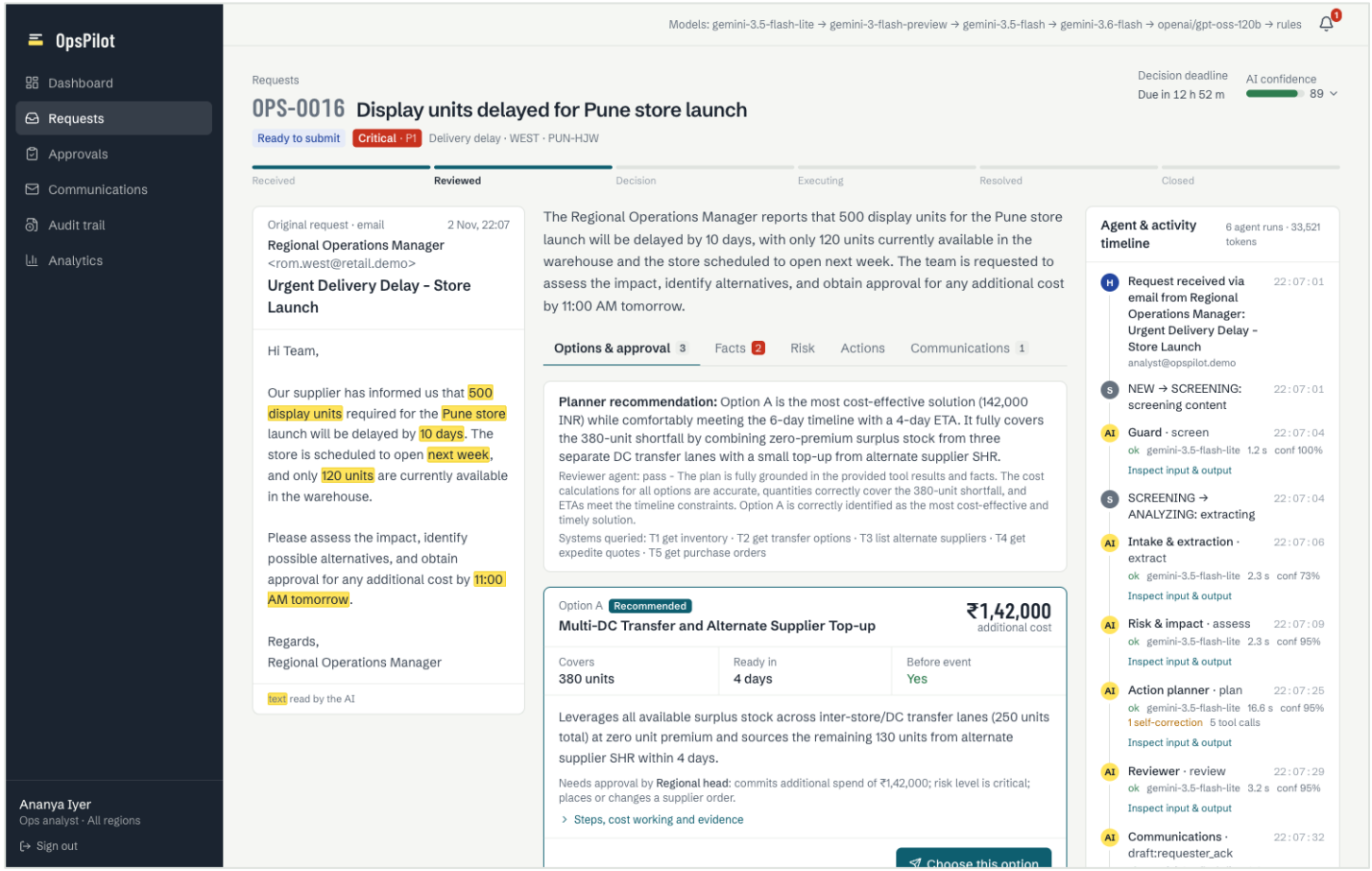
Model chain (each has its own quota pool)

1. Gemini 3.5 Flash-Lite
2. Gemini 3 Flash
3. Gemini 3.5 Flash
4. Gemini 3.6 Flash
5. Groq gpt-oss-120b
6. Deterministic rules

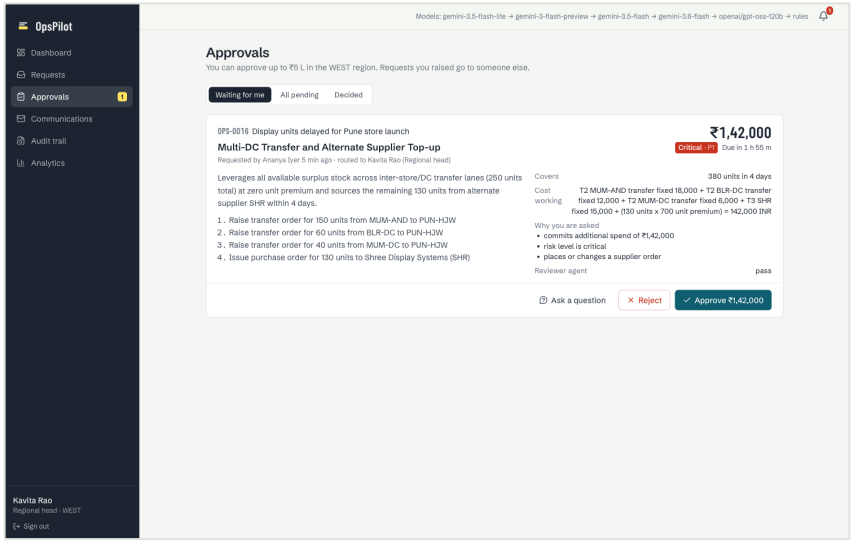
Runs on a 1 GB free VM

- systemd service, about 140 MB RAM
- Single-file database, demo reset in one click
- Docker Compose for local runs; GitHub Actions CI
- Trade-off: single instance; Postgres + job queue is the scale path

Every AI value shows where it came from and who can act on it



Request detail: highlighted evidence in the source email, facts checked against systems, three costed options with the required approver, and a live agent timeline with inspectable inputs and outputs.



Approval center: what is being approved, the cost working, why this person is asked, approve / reject with budget cap / ask a question, SLA countdown.

The video walks the assignment email from receipt to closure, then demonstrates seven edge cases live.

15 edge cases, each detected, surfaced, recovered and recorded



Data quality

- Missing info → clarification email, SLA paused
- Email 150 vs inventory 100 → human picks source
- Duplicate sender → merged, requester linked
- Multi-issue email → split into linked cases
- Wrong extraction → override, AI value kept, re-run
- Low confidence → blocked without justification



Workflow

- Approver silent → reminder, auto-escalation
- Rejection → re-plan under budget cap
- Supplier revises date → approval superseded
- Launch postponed → approvals withdrawn



Resilience

- Model or quota failure → next model, then rules
- Inventory API down → breaker, unverified flag
- Email bounce → retry ×3, dead-letter, in-app
- Agents disagree on risk → human acknowledges



Security

- Out-of-authority approval → refused + audited
- Prompt injection → flagged, quarantined, inert

Every case has an automated test (35 in total) and a simulated email or admin control for live demonstration.

Built to be trusted, not just demoed

0

agent tools that can approve, send or spend

5

model backends before rules-only mode

35

automated tests, one per edge case

100%

of actions in a hash-chained audit log



Authority in code

Role rank, region scope, approval limit and separation of duties are checked server-side; refusals explain why and are audited.



Injection defence in layers

Regex signals + Guard agent on datamarked text; untrusted mail cannot merge into or edit other cases; humans gate every effect.



Tamper-evident record

Append-only events with SHA-256 chaining; database triggers block edits; one-click verification and CSV export.



Observable AI

Per-run prompt version, model, fallback, retries, tokens and latency; override, fallback and ungrounded rates on the analytics page.

How I would deploy this with the customer

Discover

Week 0

Shadow the ops desk, collect 50–100 real requests, map the real approval matrix, baseline the metrics

Integrate

Weeks 1–2

Ops mailbox → webhook, ERP/WMS/PO adapters (or their stock sheets), SSO, policy loaded in Admin

Shadow

Week 3

AI runs on every request while people work as today; tune prompts and rules to ≥ 90% field accuracy

Pilot

Weeks 4–6

West region decides in OpsPilot; weekly override review with the ops lead; fixes shipped weekly

Hand over

Week 7+

Region-by-region rollout, Teams/Slack approvals, runbook and training so their team owns it

−60%

request-to-decision time for P1/P2

≥ 95%

approvals decided within SLA

< 10%

extracted fields corrected by people

Pilot exit targets, measured from per-state timestamps and the AI-quality panel

Risks I would manage

- Model quota or quality → five-backend chain, validators, rules floor
- Over-trust → evidence on screen; people approve every effect
- Integration delays → adapters behind one module; start from their spreadsheets